

document vectors exist in the same vector space, making them comparable.

```
# The query text. You can change this to anything you want
to search for.
query_text = "I'm looking for a tool for creating vector
embeddings."
# Encode the query text into a vector
query_vector = model.encode(query_text).tolist()
```

Perform the search: The `client.search()` method is the main function for querying your Qdrant collection. You provide it with the collection name, the query vector, and a limit to specify the number of results you want.

```
# Perform the semantic search
search_results = client.search(
collection_name=collection_name,
query_vector=query_vector,
limit=3 # Retrieve the top 3 most similar results
)
print("\nSearching for: '{query_text}'\n")
print("-" * 30)
```

The `client.search()` call sends your query vector to Qdrant. The database then efficiently finds the vectors that are closest to your query vector based on the distance metric you defined (Cosine distance in this case).

Display the results: The search results are a list of `ScoredPoint` objects, which contain the point's ID, its vector, the payload, and most importantly, a score that indicates the similarity between the query and the result. A higher score means greater similarity.

```
# Display the results
for result in search_results:
text_payload= result.payload.get('text', 'No text found')
score result.score
print(f"Score: {score:.4f} -> Text: '{text_payload}'")
print("-" * 30)
```

By accessing the payload of each result, you can retrieve the original text associated with the vector, making the results human-readable.

Beyond the basics: Scaling and optimisation

The system described above provides a solid foundation for a prototype, but an enterprise-grade RAG solution requires more sophisticated techniques to ensure high-quality, reliable results. A simplistic similarity search is often insufficient for production workloads. A high-quality RAG system must be optimised to ensure

that the retrieved information is not just similar but also highly relevant.

One critical enhancement is hybrid search, which combines the power of semantic (vector) search with traditional keyword-based search. While semantic search excels at finding conceptual matches, keyword search is essential for retrieving specific names, dates, or product codes that may not be semantically similar but are critical for an accurate response. By performing both searches and intelligently combining the results, the system can significantly improve relevance and recall.

Another advanced technique is re-ranking. After the initial retrieval, a re-ranker can be used as a post-processing step to fine-tune the order of the search results. This process, though computationally more expensive, ensures that the most relevant chunks are presented to the LLM, leading to more precise and coherent generated text.

For enterprise-level deployments, scalability and security are paramount. Open source vector databases are built to handle these challenges. Solutions like Weaviate and Milvus offer native support for horizontal scaling and can be deployed on platforms like Kubernetes to manage growing data volumes and query loads. For data management, modern vector databases support dynamic updates and metadata filtering, allowing for continuous synchronisation of the knowledge base and more precise query capabilities. Security is also a key consideration, with some open source solutions offering enterprise-grade features such as role-based access control (RBAC) to restrict sensitive information retrieval based on authorisation levels.

Empowering your enterprise knowledge base

The tools and frameworks are now in place for open source administrators and developers to build powerful, self-hosted AI applications. The RAG framework provides a clear, cost-effective, and powerful path to make large language models a truly viable asset for enterprise knowledge management. By leveraging the flexibility and performance of open source vector databases like Qdrant, Weaviate, Milvus, or even the battle-tested pgvector extension, organisations can transform their internal data into an intelligent, accessible, and continuously up-to-date resource. The blueprint has been provided, and the power to innovate is in the hands of the community. 

By: Pavan Jakati

The author has over 16 years of experience in the IT trenches and holds patents in areas like AIOps. When not architecting the next-gen AI system, he can be found tinkering in his home lab and contributing to open source projects.